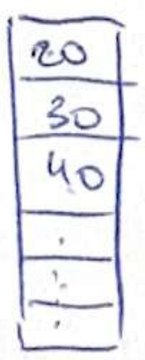
Mock-Teams

A. 1) HT, m pos, open addressing, n elem.
Search for e → is it possible that during the search more than n positions are checked?

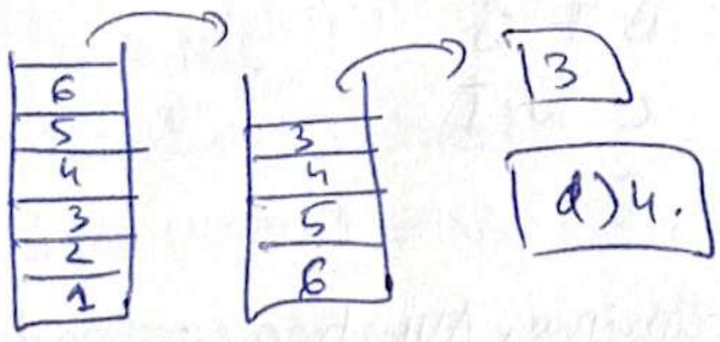
If $m > n$, yes.
In open addressing $m \geq n$. (each elem. occupies 1 slot)
If e is found ⇒ search stops.
If e is not found ⇒ search continues (exceeds n and goes to m)



search 45 ⇒ ∄ **b)** only if e not in HT.

$n = 3, m = 6$

2)



3

3) 4 6 7 + 1 - 2 5 * ++

Token	stack
4	4
6	4, 6
7	4, 6, 7 ⎫ ⇒ 6+7=13
+	4, 13 ⎭
1	4, 13, 1 ⎫ ⇒ 13-1=12
-	4, 12 ⎭
2	4, 12, 2
5	4, 12, 2, 5 ⎫ ⇒ 2*5=10
*	4, 12, 10 ⎭ ⇒ 12+10=22
+	4, 22
	26.

4) a) $O(m^4)$
d) $\Omega(m^3)$
e) $\Omega(1)$

B. 1) BT → preorder, postorder, inorder, level order.

- Preorder: ~~LRRo~~ Root-Left-Right

  DABCEHFGNI. ✓

- postorder: Left-Right-Root

  ~~EFG~~ FCBAHFNIGHD ✓

- inorder: Left-Root-Right

  BECADFHNGI ✓

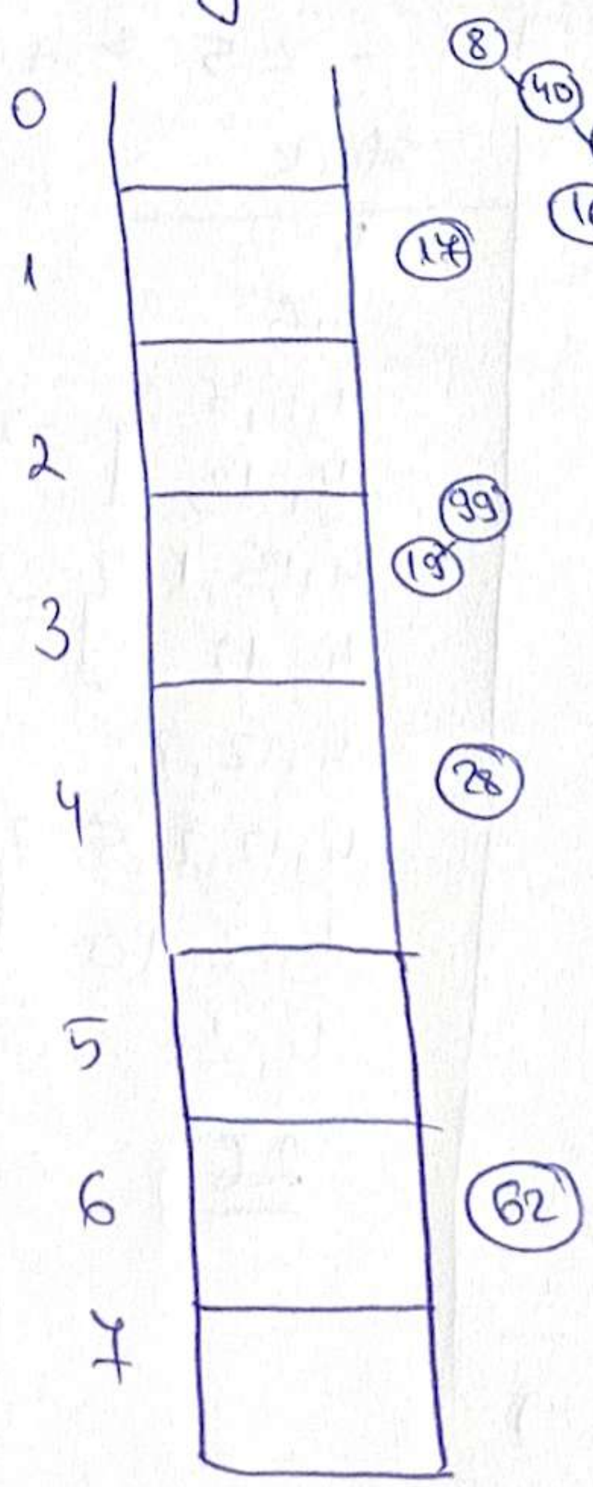
- levelorder:

  DAHBFGCNIE ~

  D
  A H
  B F G
  C N I
  E

2) HT, $m=8$ pos, div. method., sep. chaining, AVL tree ; load factor = $\frac{n}{m} = \frac{9}{8} = \alpha$

- $h(8) = 8\%8 = 0$
  $h(99) = 99\%8 = 3$
- $h(40) = 40\%8 = 0$
  $h(19) = 3$
- $h(56) = 0$
- $h(16) = 0$
  $h(17) = 1$
  $h(28) = 4$
  $h(62) = 6$



3) see mock exam (1)

Reverse SLL - in linear time
- $\Theta(1)$ E.S. complexity
- repres. of SLL
- impl.
- complexity justif.

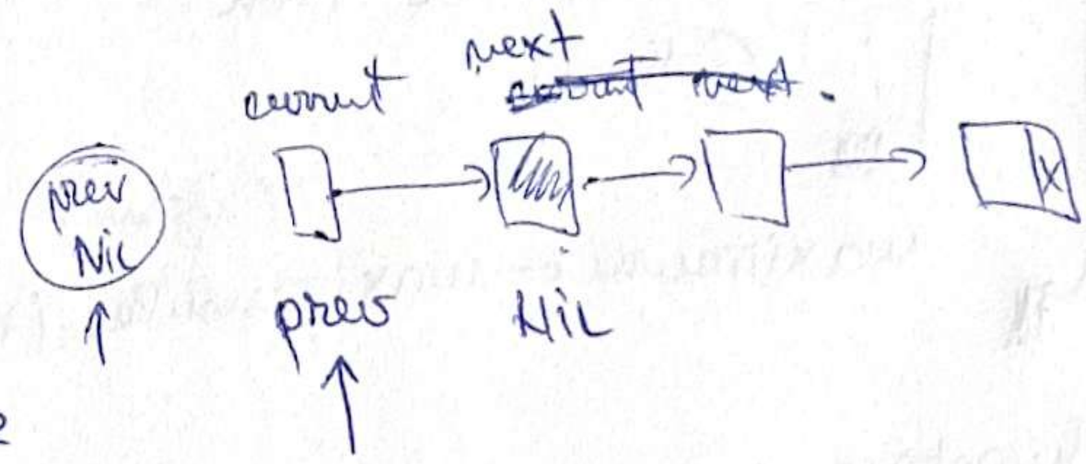
Representation:

Node:
info: TElem
next: ↑TNode

SLL:
head: ↑TNode

Implem:

```
subalgorithm reverseSLL(sll) is
    prev ← NIL
    current ← sll.head
    while current ≠ NIL execute
        nextNode ← [current].next
        [current].next ← prev
        prev ← current
        current ← nextNode
    sll.head ← prev // last visited node
```



For each node, save its old next, reverse the link, advance to next node.

$\Theta(n) = BC = WC$ because the while loop visits each node once.

ES: $\Theta(1)$ because we only use 3 pointers: prev, nextNode, current.

2) Max. value in a binary tree containing int. nr.
Give repres. of binary tree (don't memorize parent of a node)
Implem.

Complexity
Aux containers → specify

BTNode:
info: Integer
left: ↑BTNode
right: ↑BTNode

BT:
root: ↑BTNode

```
function maximum (tree) is
    if tree.root = NIL then
        @throw...
    maximum ← maximumRec (tree.root)

function maximumRec (node) is
    maxValue ← [node].info
    if [node].left ≠ NIL then
        leftMax ← maximumRec ([node].left)
        if leftMax > maxValue then
            maxValue ← leftMax
    if [node].right ≠ NIL then
        rightMax ← maximumRec ([node].right)
        if rightMax > maxValue then
            maxValue ← rightMax
    maximumRec ← maxValue
```

$Bc = \Theta(n)$
$Wc = \Theta(n)$

Total: $\Theta(n)$

Not a BST tree, so we need to visit each branch.
ES: $\Theta(h)$ from recursion stack; No aux containers.

ADT Quartiles

- init(q) - $\Theta(1)$
- add(q,e) - $O(\log_2 n)$ amortized
- getTopQuartile(q) - $\Theta(1)$ total
- deleteTopQuartile(q) - $O(\log_2 n)$ total.

PERCENTILE: → sort elems
→ the 75th percentile: elem around pos. $\lceil 0.75 \cdot n \rceil$

Choose DS:

add - $O(\log_2 n)$
get - $\Theta(1)$
deleteTop... - $O(\log_2 n)$
} ⇒ HEAP.

min-heap for top 25% (high)
max-heap for bottom 75% (low)

Representation:

Quartiles:

low : MaxHeap
high : MinHeap
size: Integer

MinHeap:

elems: Integer[]
size: Integer

MaxHeap

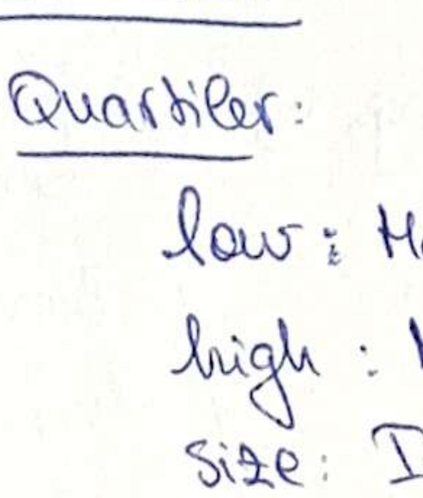
elems: Integer[]
size: Integer

Invariant

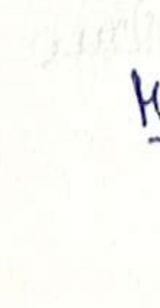
size(low) = ceil(3*n/4)
size(high) = n - size(low)

2. 10,20,30,40,50,60|70,80.

low:



high:



⇒ topQuartile = 60

3. init(q) → create both heaps, $\Theta(1)$ (MAX)

add(q,e) → If low is empty or elem ≤ root(low), add to low,
otherwise add to high.

→ Rebalance - (move to high) if low has too many elems.
- move to low if ~~high~~ low has too few elems.

→ heap add/remove = $O(\log_2 n)$ amortized.

getTopQuartile → root(low) return ⇒ $\Theta(1)$., no traversal

deleteTopQuartile → remove root from low heap ⇒ rebalance ⇒

=> move root between heaps => remove & add => $O(\log_2 n)$

$O(\log_2 n)$ + $O(\log_2 n)$

Subalgorithm deleteTopQuantile(q) is

- if q.n = 0 then → $O(\log_2 n)$
  - @throw exception
- removeMax(q.low)
- q.n ← q.n - 1
- requiredLow ← ceil(3*q.n/4)
- while q.low.size < requiredLow ex
  - x ← removeMin(q.high)
  - addMax(q.low, x)

removeMax(maxHeap)

- swap root with last elem → $O(\log_2 n)$
- size ← size - 1
- heapifyDown(root)

removeMin(minHeap) → $O(\log_2 n)$

" — "

because a Binary Heap has height $O(\log_2 n)$ and heapifyDown() traverses at most one root-to-leaf path